

2.1 Conceptos Fundamentales de Ingeniería de Requerimientos

2.1.1 Definición de Requisito

Definición formal

"A requirement is a statement which translates or expresses a need and its associated constraints and conditions. A requirement always relates to a system, software or service, or other item of interest." [1].

Explicación propia

Un requisito es la expresión formal de una necesidad que el sistema debe satisfacer, junto con las condiciones y restricciones que enmarcan esa necesidad. No se trata únicamente de una lista de funciones, sino de una declaración precisa de lo que el sistema debe hacer o cómo debe comportarse bajo circunstancias específicas. Entender correctamente qué es un requisito es el punto de partida de cualquier proceso de desarrollo, porque un requisito mal definido generará errores que se propagarán a etapas posteriores del proyecto y serán mucho más costosos de corregir. La precisión y la verificabilidad son dos propiedades esenciales: si un requisito no puede ser comprobado, no tiene utilidad práctica dentro del proceso de ingeniería.

Ejemplo aplicado al PFC

En la red social para mascotas, un ejemplo de requisito bien definido sería: "El sistema debe permitir al usuario registrar una mascota indicando nombre, especie, raza, edad en meses, peso en kilogramos, sexo y hasta cinco fotografías, completando el proceso en un máximo de tres pasos." Esta declaración expresa una necesidad concreta, incluye las restricciones relevantes y puede verificarse directamente durante las pruebas. Contrastarla con una redacción vaga como "el sistema debe registrar mascotas" evidencia por qué la definición formal del concepto importa: sin precisión, el equipo de desarrollo no sabe exactamente qué construir ni cómo comprobar si lo construyó bien.

2.1.2 Ingeniería de Requerimientos

Definición formal

"Software requirements express the needs and constraints placed on a software product that contribute to the solution of some real-world problem. The requirements process, if it is to be successful, must be considered as a process involving complex, tightly coupled activities both sequential and concurrent rather than as a discrete, one-off activity performed at the outset of a software development project." [2].

Explicación propia

La Ingeniería de Requerimientos (IR) es la disciplina que se encarga de establecer y mantener los requisitos a lo largo de todo el ciclo de vida del sistema. No es una etapa única que ocurre al inicio del proyecto, sino un conjunto de actividades continuas e interrelacionadas que van desde entender el problema del cliente hasta garantizar que lo construido corresponde a lo especificado. Su carácter interdisciplinario la distingue: involucra a ingenieros, clientes, usuarios finales y otros interesados, lo que exige habilidades técnicas y comunicativas al mismo tiempo. Una IR bien aplicada reduce el riesgo de que el sistema final no cumpla las expectativas reales, que es una de las causas más frecuentes de fracaso en los proyectos de software.

Ejemplo aplicado al PFC

En el desarrollo de la red social para mascotas, la IR se aplica desde el momento en que se estudia el problema: los dueños tienen dificultades para encontrar pareja para el cruce o para dar a sus mascotas en adopción porque deben recurrir a múltiples fuentes dispersas. A partir de ese diagnóstico, el equipo entrevista a usuarios potenciales, define qué funciones necesita la plataforma, especifica cómo debe comportarse cada una y verifica que lo construido realmente resuelve el problema original. Sin ese proceso estructurado, el sistema podría terminar con funciones que nadie usa y sin las que realmente hacen falta.

2.1.3 Características de los Requisitos Individuales

Definición formal

"An individual requirement should exhibit the following characteristics: necessary, implementation free, unambiguous, consistent, complete, singular, feasible, traceable, and verifiable." [1].

Explicación propia

Cada requisito, considerado de manera individual, debe cumplir un conjunto de propiedades para ser útil dentro del proceso de desarrollo. Debe ser necesario, es decir, que su ausencia provoque una deficiencia en el sistema; no ambiguo, de modo que todos los que lo lean lo interpreten de la misma manera; y verificable, para que sea posible demostrar mediante prueba objetiva si el sistema lo cumple o no. Además, un buen requisito no prescribe soluciones de implementación, sino que describe qué debe lograrse y deja al equipo técnico la decisión de cómo conseguirlo. Cuando un requisito viola alguna de estas características, introduce ruido en el proceso y se convierte en fuente de malentendidos, errores de diseño o pruebas imposibles de ejecutar.

Ejemplo aplicado al PFC

Considérese el siguiente requisito para la plataforma: "El sistema debe enviar una notificación al usuario propietario del perfil dentro de los 30 segundos siguientes a que otro usuario marque su mascota como favorita." Este requisito es necesario porque el dueño necesita saber que hay interés; no ambiguo porque especifica el tipo de evento y el tiempo máximo; verificable porque basta medir el tiempo de entrega de la notificación; e independiente de implementación porque no dicta si debe enviarse por correo, SMS o notificación push. Comparado con "el sistema debe notificar a los usuarios de eventos importantes", el primero es claramente superior en calidad.

2.1.4 Características de un Conjunto de Requisitos

Definición formal

"The set of requirements as a whole should exhibit the following characteristics: complete, consistent, affordable, bounded, and necessary." [2].

Explicación propia

Además de que cada requisito individualmente esté bien escrito, el conjunto completo debe poseer propiedades propias como grupo. La completitud implica que ninguna necesidad relevante quedó sin documentar; la consistencia, que no existen contradicciones entre un requisito y otro; y la acotación, que el conjunto define claramente el alcance del sistema sin incluir elementos que pertenecen a otros proyectos. La asequibilidad garantiza que el conjunto puede implementarse con los recursos disponibles, y la necesidad implica que cada requisito puede justificarse por su contribución al objetivo del sistema. Revisar el conjunto como totalidad, y no solo requisito a

requisito, es una práctica indispensable para detectar omisiones y contradicciones antes de pasar al diseño.

Ejemplo aplicado al PFC

En la red social para mascotas, el conjunto de requisitos sería inconsistente si un requisito indicara que los usuarios pueden contactarse directamente sin restricciones y otro estableciera que los datos de contacto permanecen ocultos hasta que haya una interacción previa en la plataforma: ambos no pueden coexistir sin resolver la contradicción. Por otro lado, el conjunto estaría incompleto si se definieran los flujos de cruce y adopción, pero se omitiera el proceso de reporte de publicaciones inapropiadas, que es necesario para garantizar la seguridad de la comunidad. Revisar el conjunto completo permite identificar y corregir estos problemas a tiempo.

2.2 El Proceso de Ingeniería de Requerimientos: Actividades y Fases

2.2.1 Elicitación de Requisitos

Definición formal

"Requirements elicitation is the use of systematic techniques, such as prototyping and structured surveys, to proactively identify and document customer and end user needs." [1].

Explicación propia

La elicitación es la primera actividad del proceso de IR y quizás la más crítica, porque es el momento en que el equipo de desarrollo entra en contacto directo con el problema que el sistema debe resolver. No consiste únicamente en preguntar a los usuarios qué quieren, sino en emplear técnicas sistemáticas para descubrir también las necesidades implícitas que los usuarios no verbalizan espontáneamente. Entre las técnicas más empleadas se encuentran las entrevistas estructuradas y semiestructuradas, los grupos de enfoque, la observación directa de los procesos de negocio, el análisis de documentos existentes y la construcción de prototipos rápidos. El resultado de esta fase es una primera colección de necesidades que aún no son requisitos formales, sino la materia prima que se refinará en etapas posteriores.

Ejemplo aplicado al PFC

Para la red social de mascotas, el equipo podría realizar entrevistas con dueños que hayan recurrido a grupos de Facebook o foros de criadores para encontrar pareja para sus animales, con el objetivo de entender qué frustraciones tienen con las soluciones actuales. Complementando esto con la observación de cómo publican anuncios en esas plataformas y qué información incluyen, el equipo descubre que los usuarios necesitan filtros por raza, ubicación y disponibilidad para el cruce, así como un sistema de mensajería interno que evite exponer su número de teléfono desde el principio. Estas necesidades no siempre emergen de forma explícita en las entrevistas, sino que se infieren del análisis del comportamiento real de los usuarios.

2.2.2 Análisis de Requisitos

Definición formal

"Requirements analysis is concerned with the process of analyzing the collected requirements to detect and resolve conflicts between requirements, discover the bounds of the software to be developed, and elaborate system requirements to derive software requirements." [2].

Explicación propia

Una vez recogidas las necesidades de los interesados, el análisis se encarga de transformarlas en requisitos consistentes, completos y sin ambigüedades. Durante esta etapa se detectan conflictos entre las necesidades de distintos grupos de usuarios, se determina qué cae dentro del alcance del sistema y qué queda fuera, y se priorizan los requisitos según su importancia y viabilidad. El análisis también incluye el modelado del sistema mediante herramientas como diagramas de casos de uso o modelos conceptuales, que facilitan la comunicación entre el equipo técnico y los interesados no técnicos. Una omisión habitual es saltarse esta etapa y pasar directamente de la elicitación a la especificación, lo que produce documentos llenos de contradicciones y vacíos que aparecerán como defectos en fases avanzadas del desarrollo.

Ejemplo aplicado al PFC

Durante el análisis de la plataforma de mascotas, el equipo detecta que existe un conflicto entre la necesidad de los adoptantes de ver la mayor cantidad de información posible sobre la mascota y la necesidad de los dueños de proteger su privacidad antes de establecer contacto. La resolución de este conflicto conduce a un requisito analizado: la información básica de la mascota (foto, especie, raza, edad, tamaño) se muestra públicamente, mientras que los datos de contacto del dueño solo son accesibles tras un primer intercambio de mensajes dentro de la plataforma. Este tipo de decisiones son producto del análisis y no pueden tomarse únicamente a partir de lo recogido en la elicitación.

2.2.3 Especificación de Requisitos

Definición formal

"A software requirements specification (SRS) is a structured collection of the essential requirements functions, performance, design constraints, and attributes of the software and its external interfaces; it establishes the basis for agreement between customers and contractors on what the software will and will not do." [1].

Explicación propia

La especificación es el proceso de documentar los requisitos de manera formal, precisa y estructurada, generando artefactos que servirán como referencia oficial durante el diseño, la implementación y las pruebas. El principal producto de esta actividad es la Especificación de Requisitos de Software (SRS, por sus siglas en inglés), que actúa como contrato técnico entre los interesados y el equipo de desarrollo. Una buena especificación utiliza un lenguaje claro y consistente, evita la ambigüedad y organiza los requisitos de modo que sean trazables: que pueda identificarse el origen de cada uno y verificarse si fue implementado. Existen estándares como el ISO/IEC/IEEE 29148:2018 que guían la estructura y el contenido que debe tener una especificación de calidad.

Ejemplo aplicado al PFC

La SRS de la plataforma de mascotas organizaría los requisitos funcionales en grupos como gestión de perfiles, sistema de búsqueda y filtrado, mensajería interna y gestión de publicaciones de adopción y cruce. Para cada requisito funcional se utilizaría la sintaxis EARS (Easy Approach to Requirements Syntax), por ejemplo: "The system shall allow a registered user to filter pet profiles by species, breed, age range, and location radius of up to 50 km." Esta forma de redactar asegura que el requisito sea verificable y no dé lugar a interpretaciones distintas entre el equipo de frontend, el de backend y el equipo de pruebas.

2.2.4 Validación y Verificación de Requisitos

Definición formal

"Requirements validation is the confirmation that requirements, individually and as a set, define the right system as intended by the stakeholders; requirements verification is the confirmation by examination that requirements are well-formed." [2].

Explicación propia

La verificación y la validación son dos actividades complementarias, pero conceptualmente distintas. La verificación responde a la pregunta: ¿el requisito está bien escrito? Es decir, si cumple las propiedades de calidad como no ambigüedad, verificabilidad y consistencia interna. La validación, en cambio, responde a una pregunta de mayor alcance: ¿este requisito describe realmente lo que los interesados necesitan? Un requisito puede estar técnicamente bien redactado, pero describir una funcionalidad que no resuelve el problema real del usuario; la validación es el mecanismo para detectar esa situación. Las técnicas habituales de validación incluyen las revisiones con los interesados, la construcción de prototipos o mockups y los casos de prueba de aceptación escritos a partir de los requisitos antes de comenzar la implementación.

Ejemplo aplicado al PFC

En el proyecto, la verificación implicaría revisar cada requisito de la SRS para confirmar que no contiene términos subjetivos como "rápido" o "fácil" sin cuantificarlos, que no existen dos requisitos que se contradigan y que todos tienen un identificador único para su trazabilidad. La validación, por otro lado, implicaría presentar a los usuarios un prototipo de baja fidelidad de la pantalla de búsqueda y filtrado de mascotas, y observar si los filtros disponibles corresponden a los criterios que realmente usan cuando buscan un animal. Si los usuarios descubren durante esa sesión que necesitan filtrar por estado de vacunación, ese requisito habría fallado la validación por incompleto, aun cuando estuviera bien redactado técnicamente.

2.2.5 Gestión de Requisitos

Definición formal

"Requirements management encompasses the activities that identify, document, maintain, communicate, trace, and track requirements throughout the life cycle of a system, product, or service; it includes the management of changes to the requirements and maintaining bi-directional traceability among requirements and other work products." [1].

Explicación propia

Los requisitos no son estáticos: cambian a lo largo del proyecto porque el contexto del negocio evoluciona, los interesados refinan su comprensión del problema o surgen restricciones técnicas que obligan a replantear el alcance. La gestión de requisitos es la actividad que garantiza que esos cambios se incorporen de forma controlada, sin generar caos en el equipo de desarrollo. Un elemento central de esta actividad es la trazabilidad bidireccional: poder rastrear desde cualquier requisito hacia atrás y hacia adelante. Sin gestión de requisitos, un cambio en una funcionalidad puede romper silenciosamente otra parte del sistema porque nadie tenía documentado cómo se relacionaban.

Ejemplo aplicado al PFC

En la red social de mascotas, supongamos que durante el desarrollo el equipo decide agregar un módulo de publicaciones de mascotas extraviadas, que no estaba en el alcance original. La gestión de requisitos implicaría evaluar el impacto de ese cambio:

- ¿Afecta al sistema de búsqueda y filtrado?
- ¿Requiere un nuevo tipo de perfil de mascota?
- ¿Modifica el flujo de mensajería?

Una matriz de trazabilidad que vincule los requisitos afectados con los módulos del sistema permite responder estas preguntas de forma sistemática y evita que el cambio se incorpore de manera improvisada, generando defectos que solo se descubren en las pruebas finales o, peor aún, en producción.

Referencias

- [1] “Systems and software engineering-Life cycle processes-Requirements engineering,” 2018.
- [2] “Guide to the Software Engineering Body of Knowledge A project of the IEEE Computer Society Professional Practices Committee.”